TM231

B&R Coding Guidelines

Requirements	
Training modules	TM210 – Working with Automation Studio TM223 – Automation Studio Diagnostics TM24x – At least one programming language
Software	Automation Studio 4 and above
Hardware	none

Table of contents

1 Motivation ... 5
2 Objectives ... 7
3 PLCopen guidelines ... 8
4 Reading rules ... 9
5 Rules C001-C011: Naming conventions ... 10
5.1 Rule C001: Give meaningful names to variables ... 11
5.2 Rule C002: Define a maximum length for variable names ... 12
5.3 Rule C003: Avoid using abbreviations ... 13
5.4 Rule C004: Do not reuse variable names ... 15
5.5 Rule C005: Define the use of case ... 16
5.6 Rule C006: Use prefixes for variables ... 17
5.7 Rule C007: Use suffixes for user-defined data types ... 18
5.8 Rule C008: Give meaningful names to functions and function block instances ... 19
5.9 Rule C009: Do not repeat context name in item name ... 20
5.10 Rule C010: Limit nesting of structures ... 21
5.11 Rule C011: Give meaningful names to all elements ... 22
6 Rules C101-C106: Readability ... 23
6.1 Rule C101: Define indentation ... 24
6.2 Rule C102: Define whitespaces ... 25
6.3 Rule C103: Separation of logical blocks visually ... 26
6.4 Rule C104: Define the maximum width of code ... 27
6.5 Rule C105: Define the maximum length of code block ... 28
6.6 Rule C106: Use parentheses ... 29
7 Rules C201-C205: Comments ... 30
7.1 Rule C201: Describe all elements ... 31
7.2 Rule C202: Define a comment style ... 34
7.3 Rule C203: Placement of comments ... 36
7.4 Rule C204: Headers ... 37
7.5 Rule C205: Delete commented-out code ... 38
8 Rules C301-C316: Coding practices ... 39
8.1 Rule C301: Initialize variables ... 40
8.2 Rule C302: Delete all unused variables and datatypes ... 42
8.3 Rule C303: Limit the use of global variables ... 43
8.4 Rule C304: Avoid writing to global variables from multiple tasks ... 44
8.5 Rule C305: Only call function block instances once per program cycle ... 45
8.6 Rule C306: Delete dead code ... 46
8.7 Rule C307: Avoid duplicating code ... 47
8.8 Rule C308: Avoid implicit type conversions ... 48
8.9 Rule C309: Avoid comparing floating points ... 49
8.10 Rule C310: Limit program complexity ... 50
8.11 Rule C311: Shorten Boolean assignments ... 51
8.12 Rule C312: Avoid using EDGE, EDGEPOS and EDGENEG ... 52

8.13 Rule C313: Only release code with no errors and no warnings....53
8.14 Rule C314: Do not use magic numbers....56
8.15 Rule C315: Use AND instead of nested IF/ELSE statements....57
8.16 Rule C316: Avoid Boolean comparisons in conditions....58
9 Code review....59

1 Motivation

In the field of machine automation, written code spans across:

- Teams
- Time
- Machines

Teams

Code spans across teams. During the different software development stages of the project, different people are involved. Those involved in the implementation, testing and maintenance phases. These people are the authors and readers of the written code in the project.

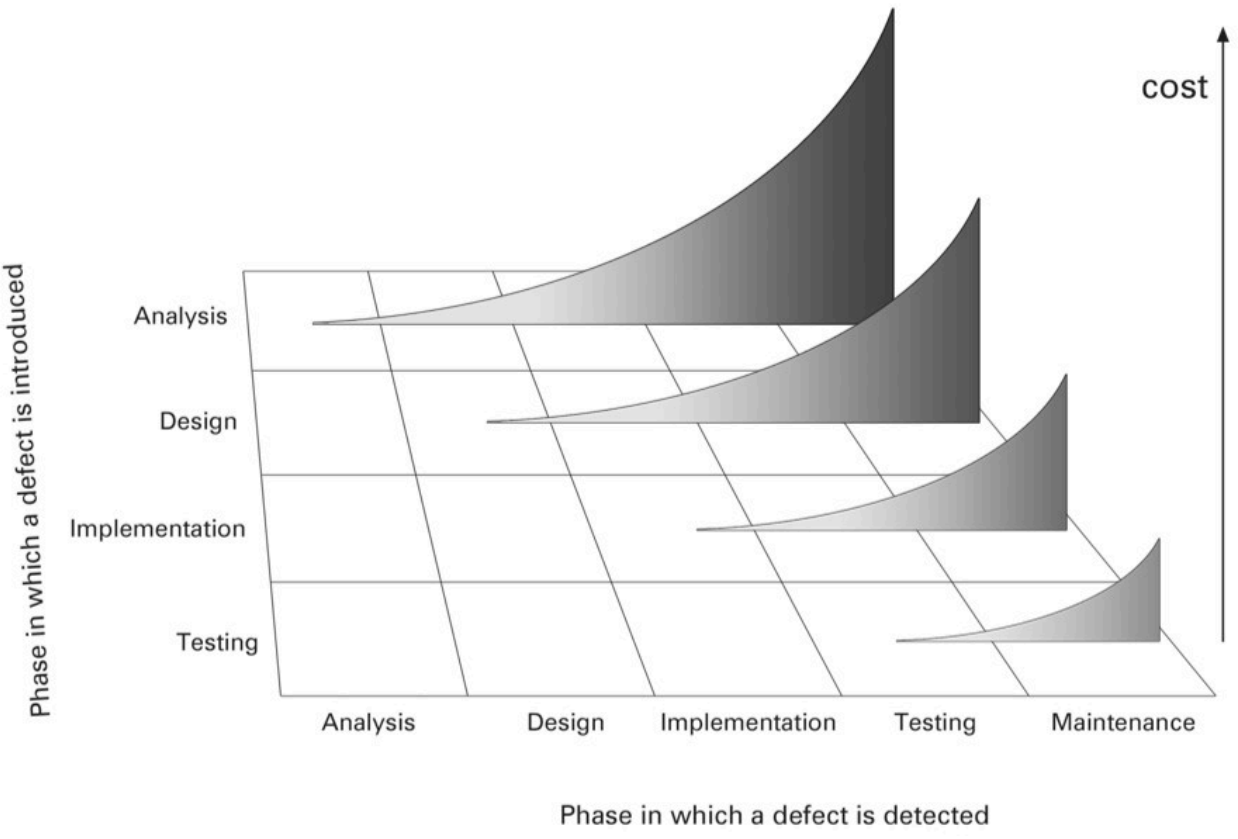
Time

Code spans across time. Different scenarios exist here. For example, a team could be currently working on writing code from scratch, on the other hand, however, the same team could also be dealing with code that was developed in the past and needs a bug fix or new feature added to it. So both “old code” and “new code” exist. Sometimes, elegantly solved issues can be reused for newer projects.

Machines

Code spans across machines. Although machines are versatile, they do share some functionality that is common. Or the same machine may also have a number of variants. As machines evolve, code should be capable evolving accordingly.

Since code spans across teams, time and machines, establishing standards for coding greatly increases quality by reducing errors while considerably decreasing the costs and downtime of the machine due to maintenance.



Coding guidelines enable clean code to be written and read by various teams at different times for several machines. The idea is to ensure consistency across these aspects. A basic indicator that the written code is clean is when it can be read like natural language.

Code which can be read with ease can be maintained with ease.

The guidelines provided here greatly contribute to increasing code readability by creating standard practices. Following these guidelines will lead to the creation of code that is readable, maintainable, reusable and with increased quality and reduced complexity.

These guidelines can be appended to existing guidelines or modified to match proved guidelines. The most important aspect, however, remains to retain consistency while following standards.

Words from the wise

What our experts have to say about the importance of clean coding...

- Programmers think they spend most of their time writing code when, in reality, they spend a lot more time reading code. Most of the time we spend reading our own code. Coding guidelines provide the ability to maintain my projects well over a long period.
- Just imagine having to read your own code in 1 year!
- Program in a way that makes it easy to be taken over by anyone.
- Time can't heal bad code!
- If code is the core business and most precious possession of a company, it should be treated as such.
- Your code is written once, but various people will read it multiple times.
- Just do it! Clean coding is the golden rule for creating software.
- Clean coding makes working in a team easier, better and more efficient.
- Clean coding helps you and your team mates create readable code and is very important as a good basis for diagnostics and for adding extra code in the future.

2 Objectives

These guidelines are intended to support the development of software technology so that software

- is easy to understand
- can be maintained with ease
- can be reused
- is of the highest possible quality

Complex technologies should be easy to understand and quick to use.

3 PLCopen guidelines

These guidelines are strongly related to existing PLCopen guidelines:
"PLCopen Coding Guidelines version 1.0"

Of course, if special PLCopen guidelines are already being followed in certain areas, e.g. motion control, safety, OPC UA, they should be given precedence.

PLCopen
for efficiency in automation

4 Reading rules

Overview of rules

These guidelines are divided into the following 4 categories:

- Naming conventions
- Readability
- Comments
- Coding practices

How to read the rules

Each rule can be identified by a "Rule ID".

Rule ID: Name of rule

Guidelines
Suggested guidelines are stated here
Examples
This section is relevant code eamples
NO:

```
code example here
```

YES:

```
code example here
```

Reasoning
Why is this guideline necessary? What are the benefits? This section contains answers to the questions.
Exception and comments
If there are any exeptions to the stated guideline or if there are any additional comments, you can find them in this section.
References
References from literature are mentioned here.

5 Rules C001-C011: Naming conventions

As was and will be repeatedly mentioned, code should look like natural language. Put simply, when you read code it should feel like you are reading a "sentence". The first and possibly most important step in this direction is how you name your variables. Ask yourself these questions while thinking of names for your variables:

- Is this name meaningful?
- Does this variable name really represent the variable's intention?
- Is this name ambiguous?
- Are there other variables in my code that look similar?
- Should the name include additional information?
- Does this name conform to accepted terminology used by the company?

5.1 Rule C001: Give meaningful names to variables

Guidelines

Use names that can be read and understood without the need for interpretation.

Examples

NO:

```
M1
```

YES:

```
InfeedMotor
```

Reasoning

Variable names should be chosen carefully. These names should clearly indicate the purpose of the variable. Avoid vague, ambiguous and generic names. Well chosen names for variables greatly boost the readability of code. It becomes easier to understand the relation between different variables in a block of code.

Exceptions and comments

Single letters like i, k, l may be used as counter variables for loops. Limit the scope of such variables to your local task.

References

- Martin, Robert C. (2009). “17: Smells and Heuristics”. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

5.2 Rule C002: Define a maximum length for variable names

Guidelines

Limit variable names to a defined maximum number of characters.

Examples

NO:

```
MaximumTemperatureForStepTuningFunctionBlock
```

YES:

```
MaxTemperatureStepTuning
```

Reasoning

The length of the variable names in the code also affects its readability. Variable names that are too short may not be sufficient to understand the purpose of the variable. On the other hand, when the code has several variables whose name are very long, it becomes difficult to read the line of code. Therefore, it is recommended to set a maximum number of characters for the variable names. In Automation Studio, variable names are limited to 32 characters. For better readability, variable names should be limited to 24 characters.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Define an acceptable name length. Identifier: Rule N6

5.3 Rule C003: Avoid using abbreviations

Guidelines

Abbreviations should be obvious and non-ambiguous. All abbreviations used in a project should be clearly documented in a glossary or table.

Examples

NO:

```
MvConv; CrBk; SftDrOp;
```

YES:

```
MoveConveyor; CraneBack; SafetyDoorOpen;
```

Reasoning

The use of abbreviations should be avoided as far as possible as they generally hinder readability.

- Abbreviations should be obvious and non-ambiguous
- All abbreviations used in the project should be clearly documented in a glossary or table

Exceptions and comments

Allowed abbreviations are listed below:

Term	Abbreviation
Absolute	Abs
Acceleration	Acc
Actual	Act
Additive/Additional	Add
Address	Adr
Advanced	Adv
Column	Col
Command	Cmd
Compensation	Comp
Configuration	Cfg
Continuous	Cont
Controller	Ctrl
Count	Cnt
Cyclic	Cyc
Deceleration	Dec
Device	Dev
Estimation	Est
Extended	Ext
Function	Fcn
Group	Grp
Identifier	Ident
Index	Idx
Interface	If
Interpolation	Ipl

Term	Abbreviation
Length	Len
Manager	Mgr
Maximum	Max
Memory	Mem
Message	Msg
Minimum	Min
Negative	Neg
Number	Nr
Numerator	Num
Object	Obj
Parameter	Par
Password	Pw
POWERLINK	PLK
Position	Pos
Positive	Pos
Reference	Ref
Relative	Rel
Source	Src
Standard	Std
Velocity	Vel
Forward	Fwd
Backward	Bwd

Table 1: Allowed abbreviation

To resolve ambiguities, e.g. pos, it must be clear from context which word is meant. Otherwise, the full name must be used.

References

- PLCopen Coding Guidelines version 1.0: Define the names to avoid. Identifier: Rule N3
- B&R Library Design guidelines V2.20

5.4 Rule C004: Do not reuse variable names

Guidelines

The following elements should not share the same name within the same scope:

- Tasks
- Programs
- Function blocks
- Functions
- Variables
- User-defined types

Examples

NO:

```
Power1; Power;
```

YES:

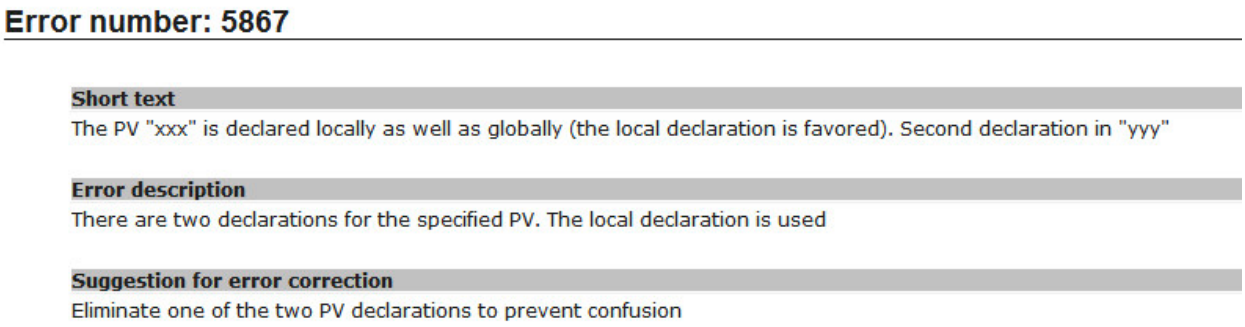
```
SortingConveyorPower; StampingConveyorPower;
```

Reasoning

Using similar names in the code or giving local and global variable the same names makes it difficult to find the variables and causes confusion during debugging or maintenance.

Exceptions and comments

If global and local variables have the same name, Automation Studio generates the following warning when compiling:



References

- PLCopen Coding Guidelines version 1.0: Different element types should not bear the same name. Identifier: Rule N9

5.5 Rule C005: Define the use of case

Guidelines

Select a style (case) for writing the names of objects.

Recommendation

- Use UPPER_SNAKE_CASE for all CONSTANTS
- Use UpperCamelCase for all other multi-word items
- Use UpperCamelCase for all file names

Examples

Item	Case	Example
Constants	UPPER_SNAKE_CASE	MAX_BOX_CAPACITY
Variables	UpperCamelCase	SortingStationConveyor
Structures	UpperCamelCase	SortingStationParameterType
Enumerations	UpperCamelCase	SortingStationStateEnum
Enumeration elements	prefix + UPPER_SNAKE_CASE	mtPULSE_MIDDLE
Function instances	UpperCamelCase	IncrementSpeed
Tasks	UpperCamelCase	LaneGates
Package	UpperCamelCase	SortingStation
mapp configuration file	UpperCamelCase	AlarmsLane.mpalarmxcore

Reasoning

Using UpperCamelCase makes multi-word names easier to read as compared to alllowercase or ALLUPPERCASE. In the case of constant names, UPPER_SNAKE_CASE is used to indicate primarily that it is a constant, and the underscore makes each word distinguishable if multiple words are used.

Exceptions and comments

When using prefixes to indicate the scope of a variable, use lowerCamelCase. For example, gAutomaticSequence. Refer to 5.6 "Rule C006: Use prefixes for variables" on page 17

References

- PLCopen Coding Guidelines version 1.0: Define the use of case. Identifier: Rule N4

5.6 Rule C006: Use prefixes for variables

Guidelines

- Use prefix + UpperCamelCase when using prefixes.
- Document the prefixes used.

Allowed prefixes are listed below:

Type	Prefix
Global	g
Digital input	di
Digital output	do
Analog input	ai
Analog output	ao
mapp Link name	g

Examples

Type	prefix
Global	gPullerControl
Digital input	diSealerCloseSensor
Digital output	doSealerOpen
Analog input	aiPullerActSpeed
Analog output	aoPullerSetSpeed
mapp Link name	gIceUserLogin

Reasoning

Prefixes are used to add more information about the variable. For example using the prefix “g” makes global variables recognizable in a program so that caution can be exercised when writing to them. However, adding too many prefixes can create complexity and long variable names.

Exceptions and comments

Using a prefix to indicate the data type of a variable (Hungarian notation) is unnecessary. This information can be obtained from variable declaration files or the tooltip. If such prefixes are used, then changing a variable's data type means that its name must be changed as well.

References

- PLCopen Coding Guidelines version 1.0: Define type prefixes for Variables.Identifier: Rule N2

5.7 Rule C007: Use suffixes for user-defined data types

Guidelines

- Structure types are written in UpperCamelCasing style and end with the suffix Type.
- Enumerations are written in UpperCamelCasing style and end with the suffix Enum.

Examples

Zone1InputsType
SortingStationStateEnum

Reasoning

The suffix makes it obvious whether the defined data type is a structure or an enumerated type.

Exceptions and comments

None

References

- B&R Library Design guidelines V2.20

5.8 Rule C008: Give meaningful names to functions and function block instances

Guidelines

- Give meaningful names

Examples

Function instance: CalculateAverageValue
Function block instance: TuneTemperatureControllerZone1

Reasoning

Function and function block instances should follow Rule C001: Give meaningful names to variables. A verb should be used as the first word in the variable name. This makes reading the code more like reading natural language.

Exceptions and comments

Using the function block name as a prefix for the function block instance should be avoided because this information is immediately available as a tool tip while hovering over the variable and using such prefixes lengthens the variable name.

References

- Martin, Robert C. (2009). “17: Smells and Heuristics”. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

5.9 Rule C009: Do not repeat context name in item name

Guidelines

Do not to repeat the context name in the item name.

Examples

- User defined types
  NO:
  ```
  ConveyorControl.Cmd.CmdStart
  ```
  YES:
  ```
  ConveyorControl.Cmd.Start
  ```
- Task names
  NO:
  ```
  project name = PaintMixer,
   task names = PaintMixerAutomatic, PaintMixerManual
  ```
  YES:
  ```
  project name = PaintMixer,
   task names = Automatic, Manual
  ```

Reasoning

Repeating names within the same context makes names unnecessarily long and provides no added advantage. It applies to any kind of hierarchical naming, including context or scope.

Exceptions and comments

None

References

None

5.10 Rule C010: Limit nesting of structures

Guidelines

Limit nesting of structures to a maximum of four levels.

Examples

NO:

```
ModuleCtrl.Puller.Para.Pid.Kp
```

YES:

```
ModuleCtrl.Puller.PidPara.Kp
```

Reasoning

Limiting the depth of nesting makes the code more readable.

Exceptions and comments

None

References

None

5.11 Rule C011: Give meaningful names to all elements

Guidelines

Use names that can be read and understood without the need for interpretation. This rule is an extension of Rule C001: Give meaningful names to variables and applies to all elements including file and folder names.

Examples

None

Reasoning

Names of elements should be chosen carefully. These names should clearly indicate the purpose of the variable. Avoid vague, ambiguous and generic names. Well chosen names for all elements greatly boost the readability and it becomes easier to understand the relation between different elements.

Exceptions and comments

None

References

- Martin, Robert C. (2009). “17: Smells and Heuristics”. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

6 Rules C101-C106: Readability

Maintainability is important in software quality, which includes readability. Good software readability decreases the cost and effort involved in maintaining code.
In addition to following the naming conventions, the visual appearance of the code determines its readability. The guidelines in this section aim at increasing code readability.

6.1 Rule C101: Define indentation

Guidelines

Define the spaces to be used for indentation and use this consistently throughout the project.

Recommendation

- Indent code with 4 spaces. (1 tab = 4 spaces)

Examples

```
CASE ConveyorStep OF
    // Power on the axis
    CONVEYOR_STEP_POWER:
        IF gConveyor.Cmd.Power AND ConveyorAxis.Info.ReadyToPowerOn THEN
            ConveyorAxis.Power := TRUE;
            gConveyor.Status.PoweringOn := TRUE;
            IF ConveyorAxis.PowerOn THEN
                gConveyor.Status.PoweringOn := FALSE;
                ConveyorStep := CONVEYOR_STEP_STANDSTILL;
            END_IF;
        END_IF;
```

Reasoning

Indenting code increases readability, especially when using conditional statements and loops. Nested statements are immediately evident.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Define indentation. Identifier: Rule L1

6.2 Rule C102: Define whitespaces

Guidelines

Define the whitespace to be used before and after logical and assignment operators and use it consistently throughout the project.

- Operators (arithmetic, relational, logical and bitwise): One whitespace before operator and one whitespace after.
- Parenthesis: No whitespace after “(“ and before “)”.
- Assignment: One whitespace before assignment and one whitespace after.
- Statement terminator: No whitespaces before and after ";".
- Function call arguments: One whitespace added after the comma to separate arguments.

Examples

NO:

```
RecipeXml.Enable        := TRUE ;
RecipeXml.DeviceName    := ADR ('Location') ;
RecipeXml.FileName      := ADR ('RecipeIcecream') ;
```

```
IF AutoMode   AND   Stop THEN
    StopMovement:=TRUE;
    SetAlarm:=TRUE;
END_IF;
```

YES:

```
RecipeXml.Enable := TRUE;
RecipeXml.DeviceName := ADR('Location');
RecipeXml.FileName := ADR('RecipeIcecream');
```

```
IF AutoMode AND Stop THEN
    StopMovement := TRUE;
    SetAlarm := TRUE;
END_IF;
```

Reasoning

Whitespace refers to the horizontal empty spaces used within a line of code. Using defined whitespaces in specified locations in the code makes the code more readable.
At first glance, adding whitespaces so that the code on the right hand side of the assignment is aligned looks aesthetically pleasing but if you add a new line of code, you might have to modify the amount of whitespace for the whole code block if the variable name is too long. Too much whitespace also creates discontinuity in the reading pattern. On the other hand, using no whitespace makes reading the code tedious.

Exceptions and comments

None

References

- Martin, Robert C. (2009). Chapter 5: Formatting. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

6.3 Rule C103: Separation of logical blocks visually

Guidelines

Define the number of empty lines to be used between logical blocks and use them consistently. Usually one empty line creates sufficient visual separation.

Examples

One empty line added after conditional statements or blocks.

```
IF NOT RecipeRegPar.Active THEN
    RecipeRegPar.Enable := TRUE;
    RecipeRegPar;
END_IF

IF RecipeRegPar.UpdateNotification THEN
    UpdateRecipeParameters;
END_IF

IF (NOT gAutomatic.Cmd.Enable) AND (NOT gAutomatic.Status.Active) THEN
    gAutomatic.Status.Stopped := TRUE;
    RETURN;
END_IF
```

Reasoning

Empty lines can be used to segregate lines of code that fit together contextually. When several blocks of code need to be distinguishable from each other, you can add a defined number of empty lines between them. Take care so as not to unnecessarily extend the vertical length of the code with too many empty lines.

Exceptions and comments

None

References

- Martin, Robert C. (2009). "17: Smells and Heuristics". Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

6.4 Rule C104: Define the maximum width of code

Guidelines

Set a limit for the maximum number of characters to be used in a single line of code. The recommended length is 120 characters.

Examples

NO:

```
IF MainControl.Cmd.Start AND InitilazedOk AND MainControl.Parameters.Automatic AND NOT MainControl.Cmd.Stop AND NOT MainControl.Cmd.Emergency THEN
```

YES:

```
IF MainControl.Cmd.Start
    AND InitializedOk
    AND MainControl.Parameters.Automatic
    AND NOT MainControl.Cmd.Stop
    AND NOT MainControl.Cmd.Emergency THEN
```

NO:

```
IF ((Axis.Command.JogPositive = FALSE) AND (Axis.Command.JogNegative = FALSE))  OR (Axis.Command.Stop = TRUE) THEN
```

YES: Logical grouping of conditional statements

```
IF ((Axis.Command.JogPositive = FALSE) AND (Axis.Command.JogNegative = FALSE))
    OR (Axis.Command.Stop = TRUE) THEN
```

Reasoning

Horizontal scrolling is undesirable. Although 80 characters is the “classic” recommended length for programs, it puts quite a limitation on the width of the code. Especially now that screen width is much larger, it is possible to adapt to more characters without sacrificing on the readability. With 120 characters, it is still possible to open 2 files side-by-side on a single monitor while using external code comparison programs. If the length of the code exceeds the defined maximum width, use new lines and consistent indentation to distribute the code across multiple lines.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Define the maximum line length. Identifier: Rule L11

6.5 Rule C105: Define the maximum length of code block

Guidelines

Set a limit for the maximum number of lines for a code block.

Examples

None

Reasoning

Like horizontal scrolling, vertical scrolling is undesirable. This is true not only in the maintenance and debugging phase but also in the implementation phase because it affects continuity while reading.
Code blocks refer to an independent block of code like states in a state machine, actions, functions, etc. Long code blocks result in constantly scrolling up or down to find the relation of the lines of code with each other. The following are tips you can use to reduce the number of lines of code:

- Split it into smaller pieces of code which focus on well defined actions or functions. Shorter code is easier to understand and test.
- Use screen height as an indicator for code length. If you have to scroll beyond the screen height to read the whole code block, then your code is too long.

Exceptions and comments

None

References

- Martin, Robert C. (2009). Chapter 5: Formatting. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

6.6 Rule C106: Use parentheses

Guidelines

Use parentheses in conditional statements to make them easier to read. Use a consistent style for placing the parentheses around blocks of code. Do not include spaces after “(“ or before “)”.

Examples

```
IF gMachine.Cmd.Power AND gMachine.Status.SwitchedOff THEN
```

OR

```
IF (Cutter.Info.PLCopenState = mcAXIS_STANDSTILL)
    OR (Cutter.Info.PLCopenState = mcAXIS_SYNCHRONIZED_MOTION) THEN
    CamAutomatGetPar.Execute := TRUE;
END_IF
```

Reasoning

Using parentheses makes your code unambiguous. Using parentheses also gives your control over the order of precedence.

Exceptions and comments

None

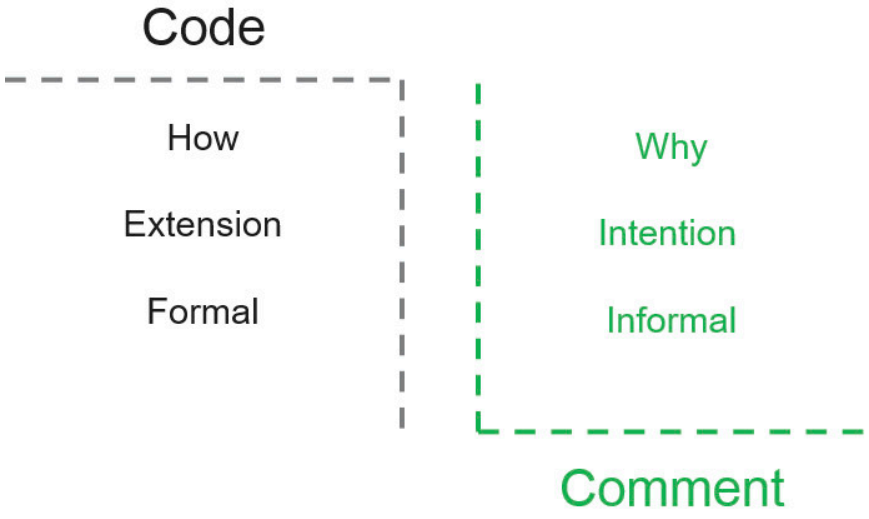
References

- PLCopen Coding Guidelines version 1.0: Use parentheses to explicitly express operation precedence. Identifier: Rule L15

7 Rules C201-C205: Comments

"Why" not "How"

A general rule to follow when adding comments to your code is to remember that that comment should express the intention of the code. How a function, feature or part of logic is implemented should be easily readable from the code itself, this is neither the role nor the purpose of the comment. Comments must reflect the intention of the author, i.e. why was this code implemented?



If you find yourself needing to write a comment, think about whether there is a way to turn it around and use the code to express the idea. Every time you use the code for expression, you should praise yourself.

Which comments make sense?

- Legal information
- Interpretation of intentions
- Interpretational comments can also be useful for explaining the meaning of certain obscure parameters or return values into a readable form. In general, a better approach is to make the parameters or return values meaningful enough by themselves. If a parameter or return value is part of a standard library or code that you can't modify, the code that helps explain its meaning proves very useful.
- Alerts: It is also helpful to use comments to warn other programmers of certain consequences.
- TODO: TODO comments explains why the implementation part of the function does nothing and what should be done in the future.

```
// TODO: Add extension to this functionality
```

- Comments can be used to magnify the importance of something that otherwise seems unreasonable.
- Comments must always be up-to-date and obsolete comments must be deleted.
- Define a fixed style for commenting. For example, use "Sentence case" for comments throughtout the project.

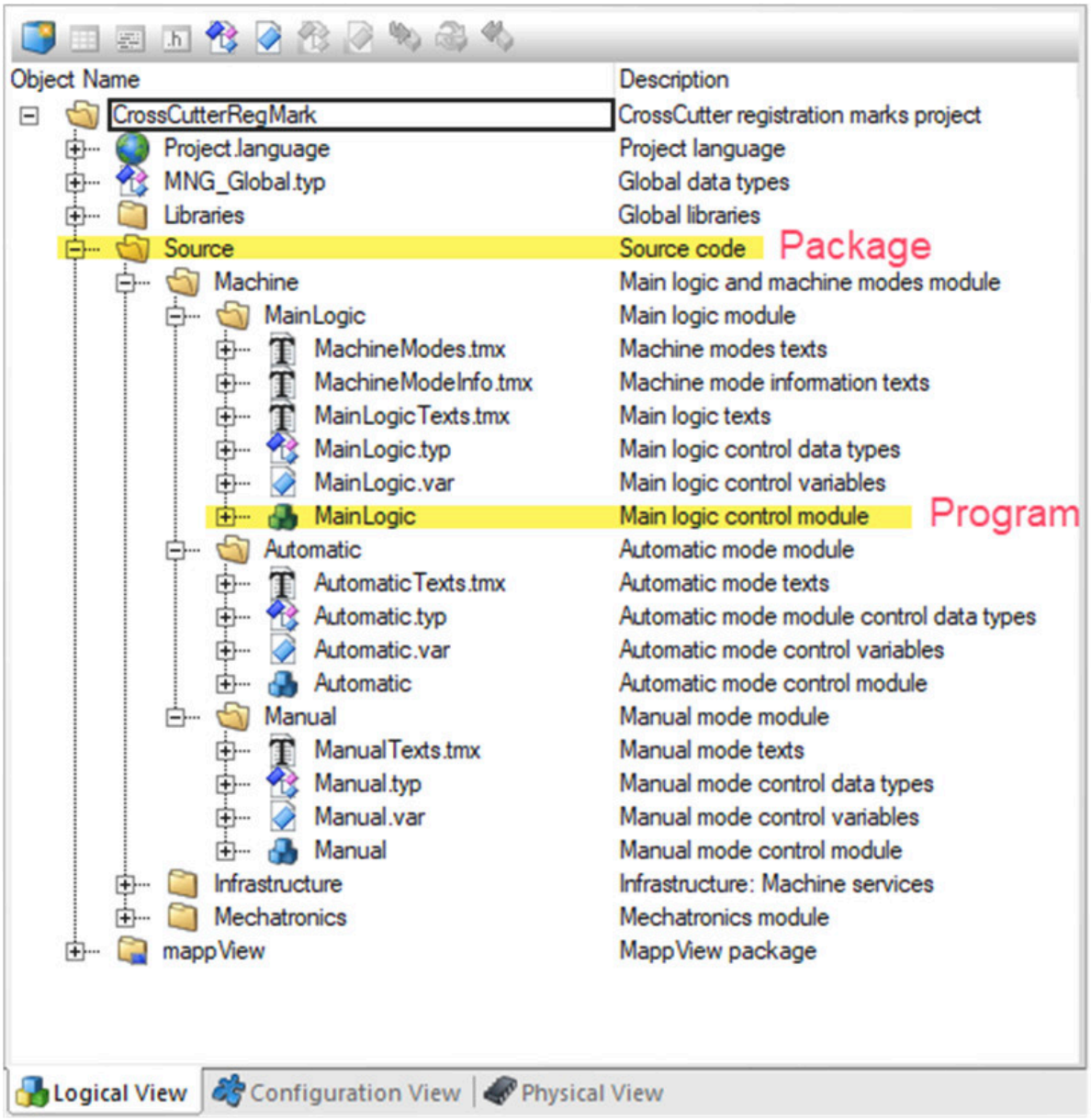
7.1 Rule C201: Describe all elements

Guidelines

Write a suitable description for all elements in an application. The description should include the purpose of the element and additional information like limits, range and units where applicable.

Examples

- Files: Programs, packages, libraries, configurations



- Variable descriptions should include valid ranges and units where applicable. Always put units of variables in square brackets [] at the beginning of the variable comment:

MAX_SHEET_WIDTH	UDINT	☑	840	[mm] Maximum sheet width
MAX_LASER_DISTANCE	UDINT	☑	700	[mm] Maximum laser distance
MAX_MARKER_DISTANCE	UDINT	☑	20	[mm] Maximum marker distance
AUTO_MODE_MIN_DECELERATION	UDINT	☑	5000	[mm/s^2] Minimum allowed deceleration in automatic mode
AUTO_MODE_MIN_ACCELERATION	UDINT	☑	5000	[mm/s^2] Minimum allowed acceleration in automatic mode
AUTO_MODE_MAX_DECELERATION	UDINT	☑	50000	[mm/s^2] Maximum allowed deceleration in automatic mode
AUTO_MODE_MAX_ACCELERATION	UDINT	☑	50000	[mm/s^2] Maximum allowed acceleration in automatic mode

- User types should include valid ranges and units where applicable

RecipeParametersType			Recipe parameters structure
Velocity	REAL	100	[mm/s] Velocity of the transported products
Acceleration	REAL	50000	[mm/s^2] Acceleration of the transported products
Deceleration	REAL	50000	[mm/s^2] Deceleration of the transported products
LaserDistance	REAL	500	[mm] Laser distance
SheetWidth	REAL	600	[mm] Sheet width
MarkerDistance	REAL	0	[mm] Marker distance

Reasoning

Describing all elements in a project also serves as documentation for the project since the description may contain additional information regarding the element that is not suitable to be added to the name of the element. For files, the accompanying description provides an overview of the file contents even before the file is opened. Depending on the type of element, the contents of the description can vary.

Exceptions and comments

When variables names are meaningful and its purpose clear, then a description of the variable becomes redundant. This is true unless the variable carries additional information like limits, valid value range and units. In this case, the description for the variable should contain this information.

References

- PLCopen Coding Guidelines version 1.0: All elements shall be commented. Identifier: Rule C2

7.2 Rule C202: Define a comment style

Guidelines

Define and use a single style for commenting throughout the project

Examples

NO:

```
(*********************************************************************
 * Copyright: B&R Industrial Automation GmbH
 * Author:    B&R
 * Created:   Dec. 1, 2020
 * Description: This program implements a conveyor as real master axis
 *********************************************************************)

PROGRAM _CYCLIC
    (* this is one style *)
    CurrentMusli.Name := UserMusli.Name;
    CurrentMusli.Quantity := UserMusli.Quantity;

    // this is another style
    FOR Index := 0 TO MAX_INGREDIENTS DO
        CurrentMusli.Ingredients[Index] := UserMusli.Ingredients[Index];
    END_FOR;

    (* and this style for block comments.
    Block comments are detailed description of functions
    or blocks of code*)
    IF ErrorStatus THEN
        ProductionState := ERROR;
    END_IF;
```

YES:

```
//*********************************************************************
// Copyright: B&R Industrial Automation GmbH
// Author:    B&R
// Created:   Dec. 1, 2020
// Description: This program implements a conveyor as real master axis
//*********************************************************************

PROGRAM _CYCLIC
    // this is one style
    CurrentMusli.Name := UserMusli.Name;
    CurrentMusli.Quantity := UserMusli.Quantity;

    // this is the same style
    FOR Index := 0 TO MAX_INGREDIENTS DO
        CurrentMusli.Ingredients[Index] := UserMusli.Ingredients[Index];
    END_FOR;

    // Block comments can also adopt this style
    // With the same consistent style for
    // each new line
    IF ErrorStatus THEN
        ProductionState := ERROR;
    END_IF;
```

Reasoning

Programming languages offer separate style for single line and multi line comments. For the purpose of maintaining consistency use either single line comment style “//” or multiple line comment style “(*.. *)” or “/* . */ ” throughout the project.
It is recommended to use the single-line comment style “//” to avoid accidentally commenting out code.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Use single line comments. Identifier: Rule C5

7.3 Rule C203: Placement of comments

Guidelines

Place single comments above the line of code. All comments describing a function can be collected together and placed above the the whole code block.

Examples

```
IF gAutomatic.Status.DistanceNextRegMarkToCutter <> 0 THEN
    // Update the distance from the next registration mark
    //  to the cutter if this is not 0 because it could happen
    // that the system switches from recoverable
    // to not recoverable while writing the cam automat parameters
    DistanceNextRegMark := gAutomatic.Status.DistanceNextRegMark;
END_IF

CASE CutterStep OF
    // Power on the axis
    CUTTER_STEP_POWER:
```

Reasoning

Placing comments on the right hand side of the code increases the width of code and limits the characters that you can actually use in that line of code. For increased readability, comments should be placed above the code it describes. This also applies to a code block that performs a specific function. In this case, it is recommended to write a comment block just above the function or code block.

Exceptions and comments

None

References

7.4 Rule C204: Headers

Guidelines

Use a simple file header, the width of the header should indicate the maximum line length (120 characters). It should contain the following information:

- File name
- Copyright
- Name of the author
- Creation date
- Short description

Examples

```
//**********************************************************************
// Copyright: B&R Industrial Automation GmbH
// Author: B&R
// Created: Dec. 1, 2020
// Description: This program implements a conveyor
//              which acts as a real master axis
//**********************************************************************
PROGRAM _INIT

    ConveyorAxis.MpLink := ADR(gConveyorAxis);
    ConveyorAxis.Parameters := ADR(ConveyorParameters);
    // Call variables/constants only used in the HMI to avoid warnings
    CallVariablesConstantsUsedOnlyHMI;
```

Reasoning

Headers should be used for customer projects. For internal projects where a version control system is used, metadata like the author's name, date of creation, date of modification, description of modification, etc. is automatically logged by the version control system. Even then, “static” information like the file description, name of the author and date of creation should be mentioned in the header. This makes it easy to get information about the file in the file itself.

Exceptions and comments

When using a version control system metadata like “Modified by” and “Modified on” is logged automatically by the tool. Therefore, it is unnecessary to add a change log to the header. This only makes the header longer and difficult to maintain.

References

None

7.5 Rule C205: Delete commented-out code

Guidelines

Delete all code that is commented out.

Examples

None

Reasoning

Commented-out code often tends to age badly in a project. Commented code is usually created in the implementation phase where authors write an alternate logic, comment out the original logic to test the alternate logic, make changes and then forget about the original logic that they commented. This way, only the author knows why the code was commented. It usually remains a mystery for other authors who work on the same project. In addition to that, it takes up valuable visual space and affects readability.

Exceptions and comments

None

References

- Martin, Robert C. (2009). "Chapter 7: "Bad Comments". Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

8 Rules C301-C316: Coding practices

"To do or not to do", that really is the question here. This section contains guidelines for "good" coding practices as well as examples for what are considered "code smells" in software development.

8.1 Rule C301: Initialize variables

Guidelines

Give initial values to variables.

Examples

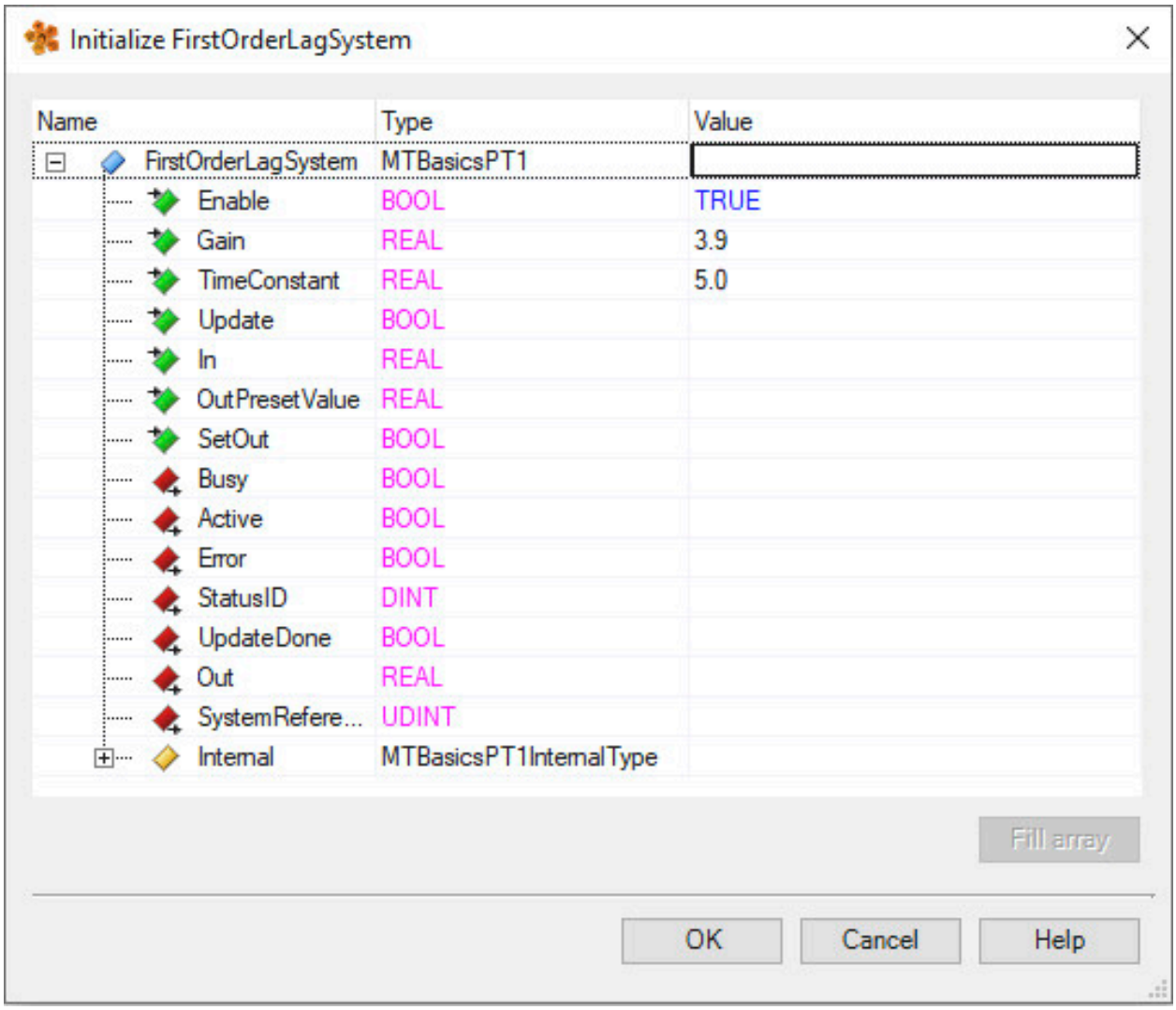
Variables (.var)

```
VAR
    SetSpeed : REAL := 0.1; (*[m/s] Desired speed *)
END_VAR
```

Types (.typ)

```
TYPE
    MusliQuantityEnum :
    (
    SMALL := 5,
    MEDIUM := 10,
    LARGE := 20,
    FAMILY := 50
    );
    MusliType :    STRUCT
    Name : STRING[80] := 'Grand old duke';
    Quantity : MusliQuantityEnum := FAMILY;
    Ingredients : ARRAY[0..MAX_INGREDIENTS]OF REAL := [2(10),2(20),30];
END_STRUCT;
END_TYPE
```

Function block instances (.var)



Reasoning

Initializing variables ensures that the code will have a "starting" value for that variable.
Variables that have a default value of 0 need not be initialized. Elements of a user defined structure should also be initialized. The benefit of this is that defined instances inherit these default values. This makes it easier and more convenient to start using the structure. This also applies to function blocks within a user library. The function block instance will inherit the configuration from the function block.

Exceptions and comments

Exceptions

- Variables that have attribute RETAIN will automatically have their values initialized to their retained value when power is reset.
- Variables that are linked to physical inputs do not need to be initialized.

References

- PLCopen Coding Guidelines version 1.0: All variables shall be initialized before use. Identifier: Rule CP3

8.2 Rule C302: Delete all unused variables and datatypes

Guidelines

All variables that are declared must be used in the project.

Examples

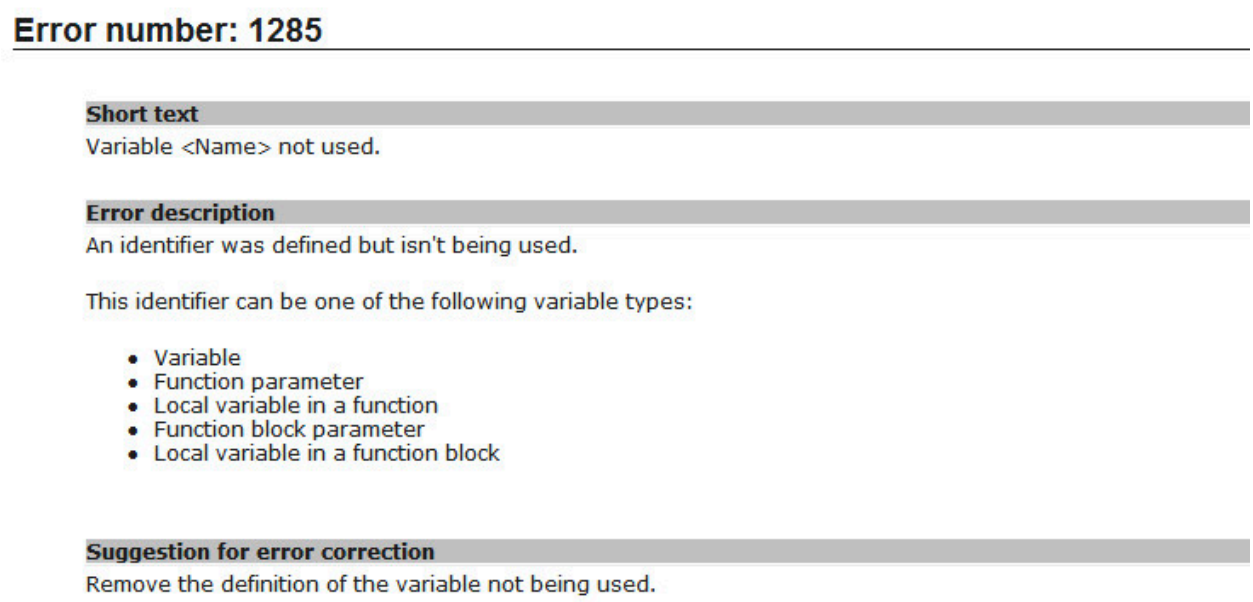
None

Reasoning

Unused variables in the project are undesirable because they cause confusion and are difficult to maintain.

Exceptions and comments

Automation Studio outputs the following error if there are any unused variables in the project.



References

- PLCopen Coding Guidelines version 1.0: Do not declare variables that are not used. Identifier: Rule CP24

8.3 Rule C303: Limit the use of global variables

Guidelines

Limit the use of global variables

Examples

None

Reasoning

The scope of variables should be decided with care. The best approach is to initially consider all variables as all "local". Then when faced with the need to share a value between programs, it should be escalated but restricted to the package-level only. The last step used only rarely is to define the variable as "global". As stated also in many web sites speaking of software design, using global variables is something that should be limited to the minimum set of variables that are absolutely necessary.

Why you should avoid using global variables

- Global variables are visible everywhere in the project
- Global variables increase the dependencies between programs
- Global variables make the project less modular
- Global variables are difficult to maintain

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Define an acceptable name length. Identifier: Rule CP18

8.4 Rule C304: Avoid writing to global variables from multiple tasks

Guidelines

Assigning values to a global variable should only be performed within one program.

Examples

None

Reasoning

As mentioned in Rule C303: Limit the use of global variables, global variables are difficult to maintain. If a global variable is being used and a value is assigned to it, this operation should be done from one program only. This makes global variables more maintainable.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: A global variable may be written only by one PROGRAM. Identifier: Rule CP26

8.5 Rule C305: Only call function block instances once per program cycle

Guidelines

Function block instances should be called only once per program cyclic irrespective whether the call is conditional. If function block instances are used in a state machine, they should typically be called only once outside the state machine.

Examples

```
CASE ControlState OF
    STEP_WAIT:
        // code for this state
    STEP_INIT:
        // code for this state
    STEP_CONTROL:
        // code for this state
END_CASE;

// Function blocks calls
ControlTemperature();

Output := ControlTemperature.Out;
```

Reasoning

This rule ensures fixed cyclic calling (and avoids double calls within one cycle), which is important for many typically used (asynchronous) function blocks, such as mapp, mappMotion, etc.

Exceptions and comments

Exceptions to this rule can only be applied if there is a necessary technical reason. Possible cases where function block instances may be called multiple times in a program cycle:

- Counter instances that need to count multiple times within a program cycle.

References

- PLCopen Coding Guidelines version 1.0: Function block instances may only be called once. Identifier: Rule CP20

8.6 Rule C306: Delete dead code

Guidelines

Delete all code that is not used.

Examples

None

Reasoning

Dead code is code that cannot be reached conditionally. There are several reasons why dead code exists in an application, the most common being that the author implemented a feature that could possibly be used in the future. Avoid this approach (YAGNI: You aren't gonna need it). Use other project management tools to document "future" features and implement them only when these are approved. Writing code that is unnecessary only increases the effort for maintenance and creates bulky code.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: All code shall be used in the application. Identifier: Rule CP2

8.7 Rule C307: Avoid duplicating code

Guidelines

Avoid writing code that is identical or that has the same outcome.

Examples

None

Reasoning

Duplicate code is code that is identical in implementation or code that achieves the same result that varies slighty in its implementation. In either case, duplicate code increases the effort to maintain a project. Changes made to one code must subsequently be made for all the duplicates. Often, this process is tedious as it is difficult to keep track of the locations of all the duplicates in the project.
Use the **Don't Repeat Yourself** (**DRY**) principle to avoid as much duplicated code as possible. Identical code in the project can be replaced with one of the following solutions corresponding to the use case:

- Functions
- Function blocks
- Actions (IEC)

Exceptions and comments

None

References

- Martin, Robert C. (2009). Chapter 17: Smells and Heuristics. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

8.8 Rule C308: Avoid implicit type conversions

Guidelines

Explicit casts should be used for all type conversions in the code.

Examples

NO:

```
VariableRealA := VariableUintB / 2;
```

When VariableUintB has value 5, the result, VariableRealA will be 2
YES:

```
VariableRealA := UINT_TO_REAL(VariableUintB) / 2;
```

When VariableUintB has value 5, the result, VariableRealA will be 2.5

Reasoning

Writing from one datatype to another should be done with caution as type conversions can lead to loss of precision or range. Where it is absolutely necessary, any type conversion in the code should be done explicitly. Explicit casts makes the code easier to understand and maintain.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Data types conversion should be explicit. Identifier: Rule CP25

8.9 Rule C309: Avoid comparing floating points

Guidelines

Instead of directly comparing floating points for equality or inequality, find the absolute difference between the 2 floating point variables and then check if this difference is greater than a threshold.

Examples

NO:

```
IF ActualTemperature = SetTemperature THEN
    // execution of this branch is sensitive to rounding errors
END_IF;
```

YES:

```
IF ABS(ActualTemperature - SetTemperature) < MIN_VALUE THEN
    // MIN_VALUE is a constant with
    //  value equal to the precision required.
END_IF;
```

Reasoning

Floating point arithmetic is complicated. Comparing floating points for equality or inequality is prone to rounding errors and often returns inconsistent values.

Exceptions and comments

Comparison operators like “>” and “<” are allowed.

References

- PLCopen Coding Guidelines version 1.0: Floating point comparison shall not be equality or inequality. Identifier: Rule CP8

8.10 Rule C310: Limit program complexity

Guidelines

Set a limit for the complexity of the code and maintain the code within this complexity limit.

Examples

Nesting of IF statements should not exceed four levels
NO:

```
IF Condition1 THEN
    IF Condition2 THEN
        IF Condition3 THEN
            IF NOT Condition4 THEN
                 TestValue := 10;
             END_IF;
        END_IF;
     END_IF;
END_IF;
```

YES:

```
ConditionOK := Condition1 AND Condition2 AND Condition3 AND NOT Condition4;
IF ConditionOK THEN
    TestValue := 10;
END_IF;
```

Reasoning

Complex code is difficult to understand and even more difficult to maintain. Complexity can be measured based on several metrics. For example, according to Thomas J. McCabe, Sr., “cyclomatic complexity” is a quantitative measure of the number of linearly independent paths through a program's source code. Complexity can be reduced by reducing the number of nested statements.

Exceptions and comments

None

References

- PLCopen Coding Guidelines version 1.0: Limit the complexity of POU code. Identifier: Rule CP9

8.11 Rule C311: Shorten Boolean assignments

Guidelines

Shorten Boolean assignments that are the result of logical operations. Simplify these by using direct assignment instead of longer IF/ELSE statements.

Examples

Example 1
NO:

```
IF gMachine.Cmd.Reset THEN
    MainUIConnect.AcknowledgeAll := TRUE;
ELSE
    MainUIConnect.AcknowledgeAll := FALSE;
END_IF;
```

YES:

```
MainUIConnect.AcknowledgeAll := gMachine.Cmd.Reset;
```

Example 2
Instead of:

```
IF MainAlarmXCore.PendingAlarms = 0 THEN
    gMachine.Status.AlarmActive := FALSE;
ELSE
    gMachine.Status.AlarmActive := TRUE;
END_IF;
```

Recommended style to reduce complexity

```
gMachine.Status.AlarmActive := (MainAlarmXCore.PendingAlarms > 0);
```

Reasoning

Simplifying by direct assignment reduces the complexity of code and makes it more readable.

Exceptions and comments

None

References

None

8.12 Rule C312: Avoid using EDGE, EDGEPOS and EDGENEG

Guidelines

Use one of the following possibilities for detecting positive or negative edges on Boolean variables:

Detecting an edge by writing the current status to a variable:

- Detecting a positive edge:

```
IF Variable AND NOT OldVariable THEN
    // positive edge detected
    // write remaining code for this condition here
END_IF;
OldVariable := Variable;
```

- Detecting a negative edge:

```
IF NOT Variable AND OldVariable THEN
    // negative edge detected
    // write remaining code for this condition here
END_IF;
OldVariable := Variable;
```

Detecting an edge using standard IEC 61131-3 functions

- The R_TRIG() function block recognizes rising edges from BOOL values.
- The F_TRIG() function block recognizes falling edges from BOOL values.

Examples

Description

EDGE/EDGEPOS/EDGENEG are functions that are easy and quick to use compared to R_TRIG or F_TRIG function blocks because it's not necessary to handle an instance. These functions compare the value passed as input with an old value that is hidden and refreshed with the function call itself. A problem could potentially occur if this function is not called cyclically but conditionally conditions: a FOR LOOP, CASE, IF condition, etc. In these cases, the old value is not refreshed, and the edge is incorrently taken into account the next time.

Exceptions and comments

The only recommended way to call these functions is to call them cyclically. A good practice is allocating a local Bool.

```
PositiveEdgeVariable := EDGEPOS(Variable);
```

References

None

8.13 Rule C313: Only release code with no errors and no warnings

Guidelines

Code should be released with no compiler errors or warnings.

Examples

Frequent compiler warnings:

- 1281: <Name> signed/unsigned mismatch. Certain operations (multiplication, division, modulo division and comparisons) return different results for signed operands than for unsigned operands.
- 1292: Implicit conversion from <DataType1> to <DataType2>: Possible data loss Implicit conversion from <DataType1> to <DataType2>: Possible data loss
- 5867: The PV "xxx" is declared locally as well as globally (the local declaration is favored). Second declaration in "yyy".
- 5874: Variable name is declared but not used in the current configuration.

Reasoning

Compiler warnings should not be ignored as they could lead to unexpected and unwanted behavior on the PLC. Resolving compiler warnings can help avoid potential bugs.

Exceptions and comments

Some warnings are unavoidable such as:

- Licensing warnings

Error number: 447

Short text

The project contains components that require licensing. Open the B&R Technology Guarding tool for more information.

Error description

The configuration contains components that require licensing. This is a warning that is generated any time such components are used in a project. The project is otherwise able to be compiled. Once the respective components have been licensed, the project can be run on the target system without any restrictions.

Suggestion for error correction

For more information on components that require licensing, see the B&R Technology Guarding tool. Select **Tools** - **Technology Guarding** from the menu to open it.

- Additional files not part of project

Error number: 9232

Short text

Additional file <Name> found, which is not part of the project.

Error description

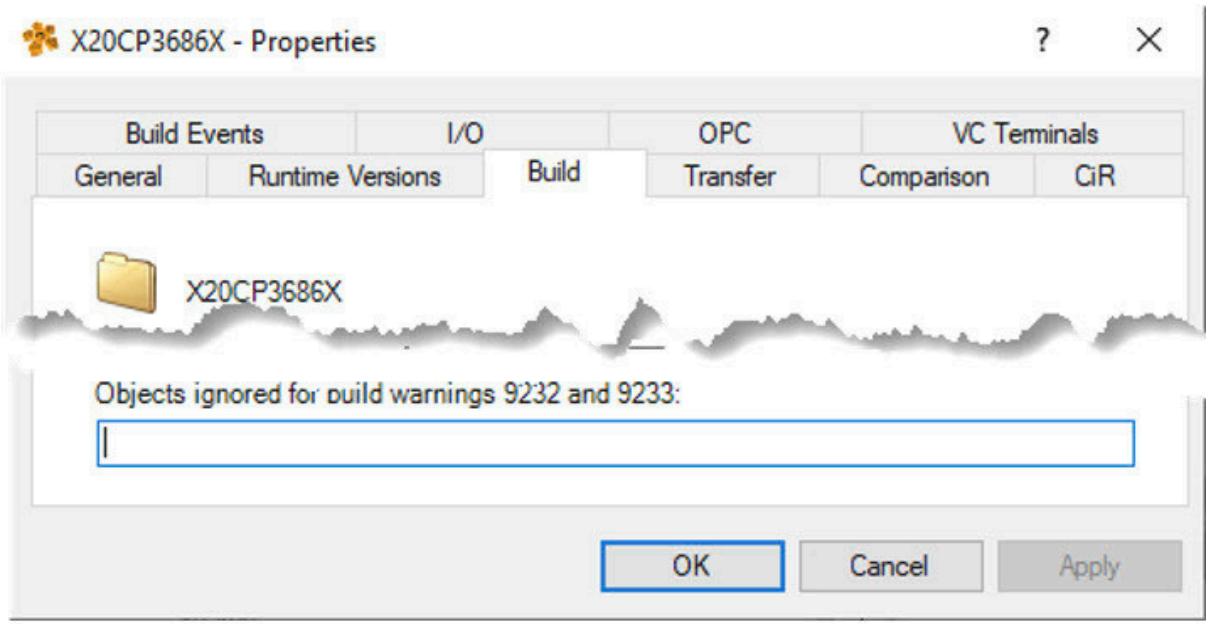
The project contains a file that is not entered in the project structure. If this file is an ANSI C header file, errors may occur when building the project if this file is being included in a program or library. Although it may be possible to compile the project without generating any error messages, the "incorrect" header file will be used, which will result in variables and data types having incorrect values or sizes.

Suggestion for error correction

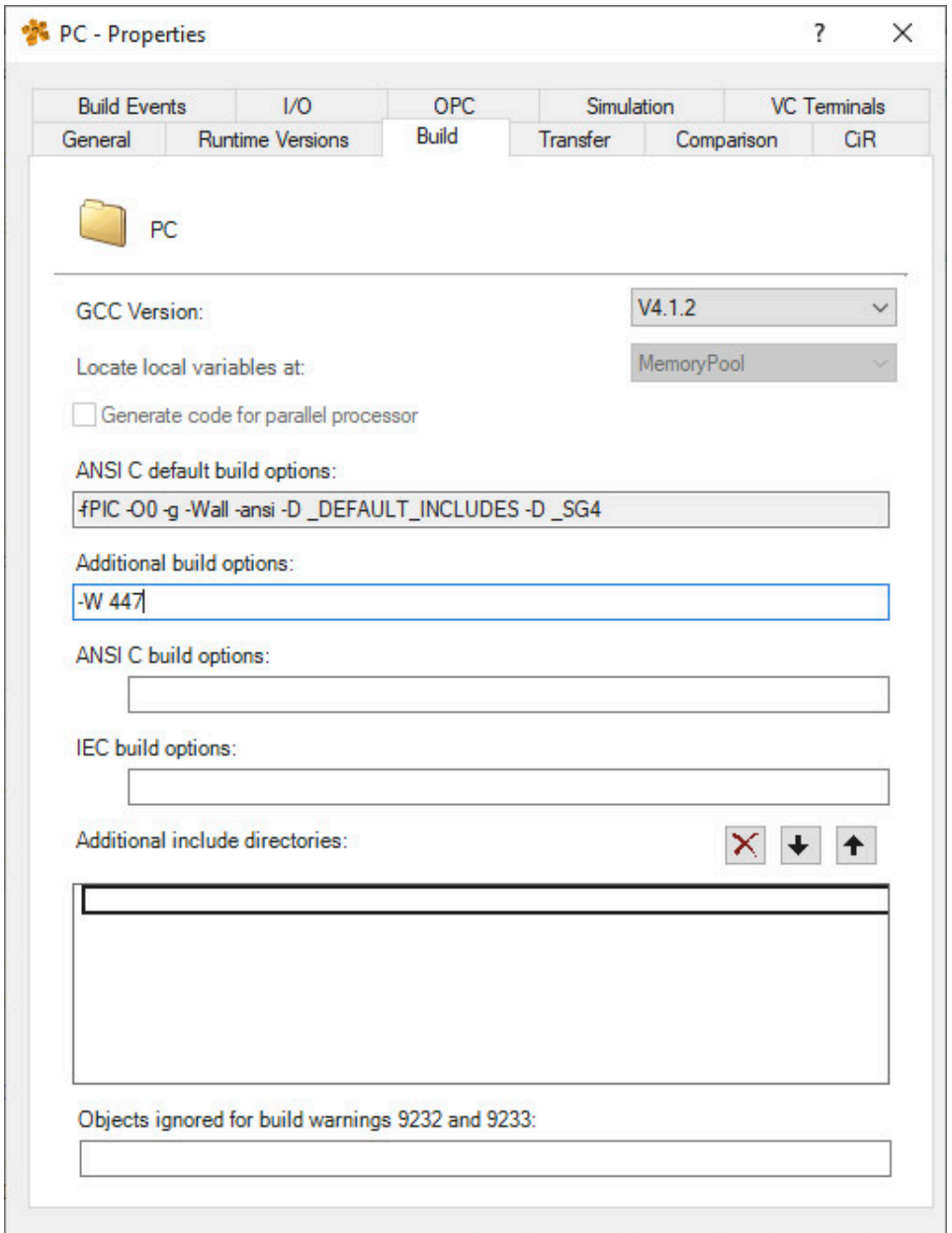
Double-clicking on the error message selects the object where the directory is located in the logical project view. If this file is an ANSI C header file, you'll have to make sure that the file isn't in the project by mistake. The file should either be removed or put into the project structure.

In Automation Studio, these warnings can be suppressed using the following compiler option:

- Ignore objects for warnings 9232 and 9233



- -W <error_number1> <error_number2> ...



Exceptions and comments

None

References

Automation Help V4.12.2.65

8.14 Rule C314: Do not use magic numbers

Guidelines

Avoid using numbers directly in your code. Replace them either with well named constants or variables.

Examples

NO:

```
CanWeight := CanWeight + (2.3 * OutletFlowRate * CycleTime);

FOR InletTankID:= 0 TO 4 BY 1 DO
    OneInletValveOpen := Inlets[InletTankID].Valve OR OneInletValveOpen;
END_FOR;
```

YES

```
CanWeight := CanWeight + (PAINT_DENSITY * OutletFlowRate * CycleTime);

FOR InletTankID:= 0 TO MAX_NO_OF_TANKS BY 1 DO
    OneInletValveOpen := Inlets[InletTankID].Valve OR OneInletValveOpen;
END_FOR;
```

Reasoning

- Increased readability: When you use a magic number, the intent or meaning of that number is disguised and may be unclear. Replacing it with a constant allows the reader to think about the knowledge the number represents rather than worrying about the number itself. Consequently, code gets more self-explanatory and therefore more readable.
- Increased refactorability: Avoiding magic numbers makes code easier to refactor and less prone to bugs. It helps us keep the DRY (**D**on't **R**epeat **Y**ourself) principle. If the value of a magic number ever changes, you would have to find all instances of the number in the project and replace them. Having a constant, you only have to change it in one place.

Exceptions and comments

None

References

- Martin, Robert C. (2009). Chapter 17: Smells and Heuristics G25. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. ISBN 978-0-13-235088-4.

8.15 Rule C315: Use AND instead of nested IF/ELSE statements

Guidelines

Use AND instead of nested IF/ELSE statements

Examples

NO:

```
IF ConveyorStep <> STATE_CONVEYOR_INIT THEN
    IF ConveyorStatus <> ERR_OK THEN
            ConveyorStep := STATE_CONVEYOR_ERROR;
    END_IF
END_IF
```

YES:

```
IF (ConveyorStep <> STATE_CONVEYOR_INIT)
   AND (ConveyorStatus <> ERR_OK) THEN
    ConveyorStep := STATE_CONVEYOR_ERROR;
END_IF
```

Reasoning

Increases readability.

Exceptions and comments

None

References

None

8.16 Rule C316: Avoid Boolean comparisons in conditions

Guidelines

If a Boolean variable should be checked in a condition, an explicit comparison with TRUE or FALSE should be avoided since it unnecessarily lengthens the code.

Examples

NO:

```
IF Enable = TRUE THEN
IF Ready = FALSE THEN
```

YES:

```
IF Enable THEN
IF NOT Ready THEN
```

Reasoning

The IF statement evaluates a Boolean expression. The Boolean expression evaluates to either TRUE or FALSE. For better readability, it is recommended to directly use Boolean variables in the condition without explicitly comparing them to values TRUE or FALSE.

Exceptions and comments

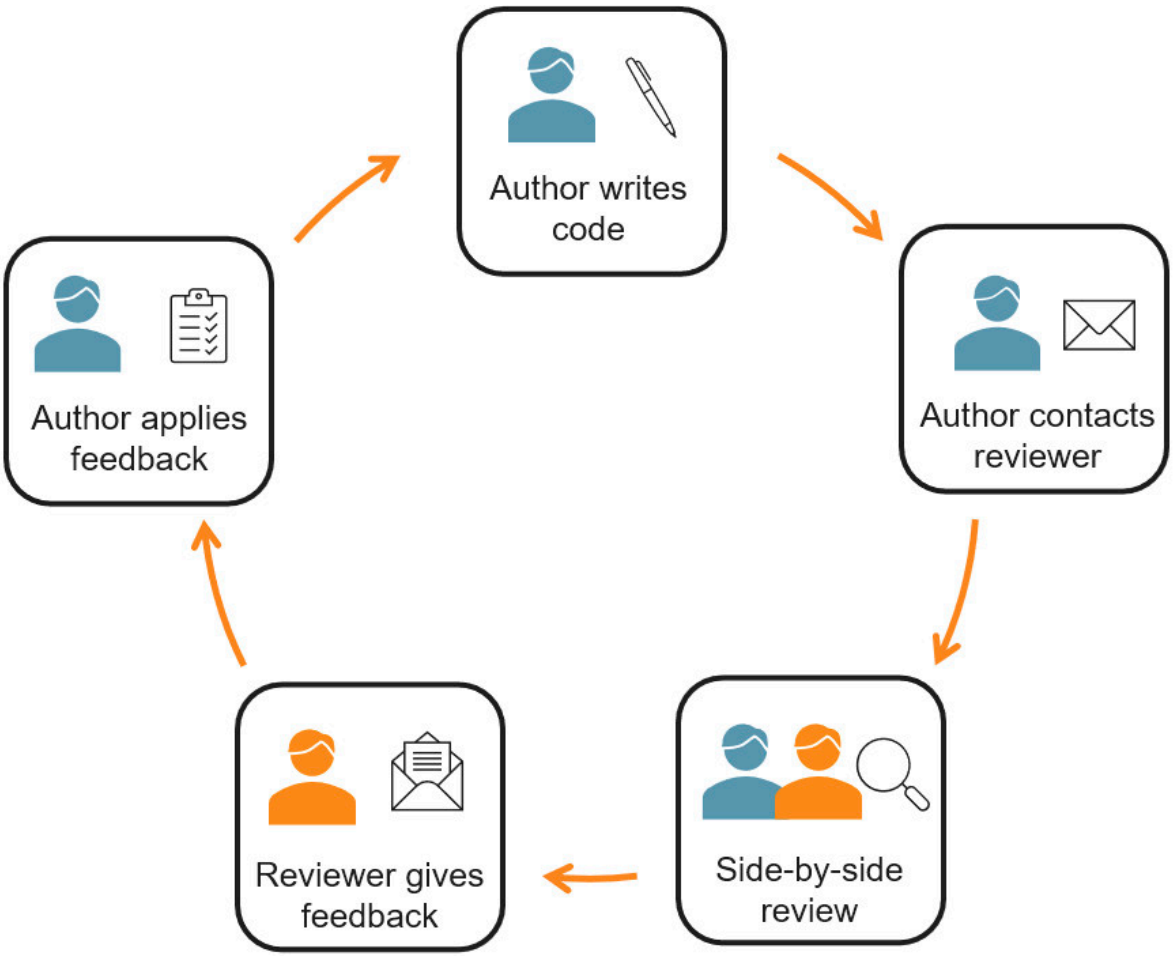
None

References

None

9 Code review

By following these guidelines, you have ensured that your code is readable...for you! But is it readable for others, too? One very useful practice to increase readability and in turn increase the quality of your code is a "code review".
To err is human! So, even though you've followed these guidelines meticulously, there could still be some inconsistencies which you don't notice or there may be a more efficient way to implement a particular routine. Code review comes in quite handy as a means to catch these inconsistencies and possibly provide suggestions to increase code quality.
How does it work? Code review basically means having someone else check your code. In the software development cycle, code review can be integrated into the "implementation" phase. This means you implement a block of code (functionality/feature/bug-fix) and have it reviewed by a colleague. After the review, make the necessary changes and move on to the next phase of development. Alternatively, you can implement part of the functionality, have it reviewed and then continue on to implementing the next part and so on.



What are the benefits of reviewing code?

Code review is a tried and tested method with several advantages.

Benefits

- Reinforces coding guidelines
- Qualitative measure of readability
- Opportunity to learn new practices
- Provides a platform for knowledge transfer
- Provides alternate solutions to problems
- Encourages team work

What are good code reviewing practices?

- Code review should not take more than 60 minutes for about 200 to 400 lines of code
- Authors should test their project before the code review
- Use checklists provided
- The goal is to improve quality
- Give feedback that helps not hurts
- Don't change something just for the sake of changing

Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!

Classroom Learning

Virtual Classroom Learning

Online courses

Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of learning content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

Virtual classroom

Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a selective choice of information to be used as a reference tool.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.
Access additional information here:
https://www.br-automation.com/de/academy/

Enjoy your next training course!

AutomationAcademy

B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

TM231TRE.001-ENG